

Conexão Solidária

Justificativa de escolha dos bancos de dados

A plataforma “Conexão Solidária” segue o princípio de *database per service*: cada microsserviço é dono do seu próprio banco, e a escolha da tecnologia de persistência é feita por contexto, não de forma única para todo o sistema. Dois bancos foram usados: **SQL Server** (relacional) e **MongoDB** (documentos). Este documento justifica por que cada um foi escolhido para o contexto em que está aplicado.

1. SQL Server — Usuários, Campanhas e Doações

Usado em dois serviços: **F5-CS-UsersApi** (UsersDB) e **F5-CS-CampanhasApi** (CampanhasDB, com as tabelas de Campanhas e Doações). O DoacaoWorker também acessa o CampanhasDB, mas não é dono do schema.

Por que relacional faz sentido aqui:

- **Dados estruturados e estáveis:** usuário, campanha e doação têm schema conhecido e fixo desde o início (nome, e-mail, CPF, título, datas, valores) — não há ganho em flexibilidade de schema que o documento ofereceria.
- **Integridade referencial de verdade:** uma Doação só existe se referenciar uma Campanha válida e um Doador válido; o SQL Server garante isso com chaves estrangeiras no banco, sem depender só da aplicação.
- **Consistência (ACID) em valores financeiros:** o recalcule do *ValorArrecadado* de uma campanha (feito pelo DoacaoWorker, somando as Doações num único UPDATE com subquery) precisa ser atômico e correto mesmo com doações concorrentes — exatamente o cenário em que um banco relacional com transações é mais simples e mais seguro que um documento eventualmente consistente.
- **Consultas relacionais naturais:** “listar campanhas ativas com valor arrecadado” ou “doações de um doador” são consultas de junção/agregação simples em SQL, sem precisar desnormalizar dados entre coleções.

2. MongoDB — Feedback do doador

Usado em um único serviço: **F5-CS-FeedbackApi** (FeedbackDB, coleção *feedbacks*). Cada doação pode gerar um feedback do doador sobre a experiência (rapidez, dificuldade, intenção de doar novamente, comentário livre).

Por que documento faz sentido aqui:

- **Schema com maior chance de mudar:** as perguntas de um formulário de feedback (escalas, novas perguntas, campos opcionais) tendem a evoluir com o produto; um documento JSON absorve essa variação sem migração de schema, diferente de uma tabela relacional rígida.
- **Documento auto-contido, sem necessidade de joins:** um feedback é lido e escrito como uma unidade completa (Id da doação, da campanha, do doador e as respostas) — não precisa ser normalizado em várias tabelas nem cruzado com outras entidades no próprio banco.

- **Consistência forte entre serviços não é necessária aqui:** a validação de que a doação pertence ao doador é feita de forma síncrona contra a CampanhasApi no momento do envio (via *GET Doacao/MinhasDoacoes*) — o FeedbackDB não precisa de transação distribuída com o banco relacional, só grava o resultado depois da validação.
- **Isolamento de carga de leitura/escrita:** feedback é um fluxo desacoplado do fluxo financeiro (campanha/doação); mantê-lo em outro banco evita que relatórios agregados de feedback (média de rapidez, % que pretende voltar) concorram com as operações transacionais de doação no mesmo banco.

3. Comparação resumida

Critério	SQL Server (Users, Campanhas, Doações)	MongoDB (Feedback)
Natureza do dado	Estruturado, relacional	Documento, semi-estruturado
Estabilidade do schema	Fixo, conhecido a priori	Pode evoluir sem migração
Necessidade de integridade referencial	Alta (FKs entre usuário, campanha e doação)	Baixa (documento auto-contido)
Necessidade de transação /ATOMICIDADE	Alta (recalculo de valor arrecadado)	Baixa (gravação unitária pós-validação)
Tipo de consulta típica	Junções e agregações financeiras	Leitura/escrita de documento único

4. Conclusão

A escolha não foi “SQL vs. NoSQL” como decisão única de plataforma, e sim uma decisão por contexto delimitado (*bounded context*), coerente com a arquitetura de microsserviços: o núcleo transacional financeiro (usuários, campanhas, doações) usa um banco relacional pela necessidade real de integridade e atomicidade; o feedback, um dado de leitura/escrita simples e com schema mais fluido, usa um banco de documentos. Essa polyglot persistence evita forçar um modelo de dados inadequado a um dos dois contextos.